

INavigationAware 전체 흐름

CancellationTokenSource / CancellationToken 사용법 및 역할
구체적이고 디테일하게 설명

● 전체 구조 요약

- **TitleViewModel** → 토글 버튼 클릭 → MenuView Show/Hide
- **MenuViewModel** → 메뉴 클릭 → ContentRegion에 각 View 로드
(예: HomeView, CustomerView...)
- 각 ContentView에는 해당 ViewModel이 존재 (**CRUD**)
- ViewModel 들은 **INavigationAware** 를 상속받아 초기화/파라미터처리/정리 처리

● INavigationAware 기본 구조 다시 확인

```
public interface INavigationAware
{
    // 뷰로 이동했을 때
    void OnNavigatedTo(NavigationContext navigationContext);
    // 뷰 재사용 여부 결정
    bool IsNavigationTarget(NavigationContext navigationContext);
    // 다른 뷰로 이동할 때
    void OnNavigatedFrom(NavigationContext navigationContext);
}
```

I TitleViewModel → MenuView 연결 (토글 버튼으로 Show/Hide)

● 동작 시나리오

- TitleViewModel 에서 토글 버튼 클릭 시 MenuView를 MenuRegion에 내비게이션
- 이때도 내부적으로 **MenuViewModel.OnNavigatedTo()** 호출 가능함

```
public class TitleViewModel : BindableBase
{
    private readonly IRegionManager _regionManager;
    private bool _isMenuVisible = true; // IsBusy 사용
    public DelegateCommand ToggleMenuCommand { get; }
```

```

public TitleViewModel(IRegionManager regionManager)
{
    _regionManager = regionManager;
    ToggleMenuCommand = new DelegateCommand(ExecuteToggleMenu);
}

private void ExecuteToggleMenu()
{
    _isMenuVisible = !_isMenuVisible;
    if (_isMenuVisible)
    {
        // 메뉴를 "MenuRegion"에 로드 = 메뉴 열기
        // RequestNavigate(...) -> 지정한 View(MenuView)를 Region에 로드
        _regionManager.RequestNavigate("MenuRegion", nameof(MenuView));
    }
    else
    {
        // 메뉴를 "MenuRegion"에서 제거 = 메뉴 닫기
        var region = _regionManager.Regions["MenuRegion"];
        region.RemoveAll(); // 해당 Region의 View 모두제거 -> 메뉴 숨김 효과
    }
}
}

```

II MenuViewModel → ContentRegion 네비게이션

● 시나리오

- ListBox 또는 버튼 클릭으로 메뉴 선택
- 선택된 메뉴에 따라 ContentRegion에 View (예: HomeView, CustomerView...) 로드
- 해당 ViewModel의 **OnNavigatedTo()** 호출 → 초기화

```

public class MenuViewModel : BindableBase
{
    // Prism의 RegionManager: 화면을 동적으로 전환하는 핵심 서비스
    private readonly IRegionManager _regionManager;

    // 메뉴 항목 리스트 (예: 홈, 고객, 사용자, 직원)
    public ObservableCollection<string> MenuItems { get; } = new()
    {

```

```

        "홈", "고객", "사용자", "직원"
    };

    // 현재 선택된 메뉴
    private string _selectedMenu;
    public string SelectedMenu
    {
        get => _selectedMenu;
        set
        {
            // 선택된 메뉴가 변경될 경우
            if (SetProperty(ref _selectedMenu, value))
            {
                // 해당 메뉴에 대응되는 View로 화면 전환 실행
                NavigateToContentView(_selectedMenu);
            }
        }
    }

    // 생성자에서 RegionManager 주입
    public MenuViewModel(IRegionManager regionManager)
    {
        _regionManager = regionManager;
    }

    // 메뉴 클릭 시 실행: 선택된 메뉴 이름에 따라 View 이름 매핑 및 네비게이션 실행
    private void NavigateToContentView(string menu)
    {
        // 메뉴 문자열 → View 이름 매핑
        string viewName = menu switch
        {
            "홈" => nameof(HomeView),
            "고객" => nameof(CustomerView),
            "사용자" => nameof(UserView),
            "직원" => nameof(EmployeeView),
            _ => nameof(HomeView) // 기본값
        };

        // 네비게이션 파라미터 전달 가능 (선택 사항)
        var parameters = new NavigationParameters
        {

```

```

        { "FromMenu", menu }
    };

    // ContentRegion에 View 로드
    _regionManager.RequestNavigate("ContentRegion", viewName, parameters);
}
}

```

Dictionary<string, string> 이방식도 가능

```

private static readonly Dictionary<string, string> _menuToViewMap = new()
{
    ["홈"] = "HomeView",
    ["고객"] = "CustomerView",
    ["사용자"] = "UserView",
    ["직원"] = "EmployeeView"
};

```

```

string viewName = _menuToViewMap.TryGetValue(menu, out var name)
    ? name : "HomeView";

```

이 방식은 조건이 10개 이상 또는 동적 구성시
데이터 중심으로 관리, 유지보수 쉬움

II 각 Content ViewModel에서 INavigationAware 사용법 (CRUD 포함)

```

public class CustomerViewModel : BindableBase, INavigationAware
{
    public string Title { get; set; }
    public ObservableCollection<Customer> Customers { get; } = new();

    public void OnNavigatedTo(NavigationContext navigationContext)
    {
        // 내비게이션 파라미터로 어떤 메뉴에서 왔는지 확인 가능
        // ContentRegion 에 이 View가 로드될 때 호출
        if (navigationContext.Parameters.TryGetValue("FromMenu", out string menu))
        {
            Title = $"{menu} 화면 로드됨";
        }
        // 고객목록 초기 로딩 (DataGrid, ListView 등)
        LoadCustomers();
    }
}

```

```

public bool IsNavigationTarget(NavigationContext navigationContext)
{
    // 이 뷰모델 재사용 여부 (파라미터에 따라 새 인스턴스 생성 가능)
    // true -> 기존 인스턴스 재사용, false -> 새 인스턴스 생성
    return true; // 이 View가 열려 있으면 재사용함 (기본값)
}

public void OnNavigatedFrom(NavigationContext navigationContext)
{
    // 다른 View로 넘어가기 직전 호출
    SaveUnsavedChanges(); // 필요시 데이터 정리
}

private void LoadCustomers()
{
    // 실제 API 또는 DB 로딩 로직
    Customers.Clear();
    Customers.Add(new Customer { Name = "홍길동", Age = 30 });
}

private void SaveUnsavedChanges()
{
    // 예: 저장 안된 내용 자동 저장 등
}
}

```

OnNavigatedFrom -> 데이터정리 변경사항 통보 메시지

```

public class CustomerViewModel : BindableBase, IConfirmNavigationRequest
{
    private bool _hasUnsavedChanges = true; // 예시

    public void ConfirmNavigationRequest(
        NavigationContext navigationContext, Action<bool> continuationCallback)
    {
        if (_hasUnsavedChanges)
        {
            // 메시지 박스 띄우기 (WPF 예시)
            var result = MessageBox.Show(
                "저장하지 않은 변경사항이 있습니다. 그래도 나가시겠습니까?",
                "경고",

```

```

        MessageBoxButton.YesNo,
        MessageBoxImage.Warning);
// 사용자가 Yes를 클릭하면 true, 아니면 false 전달
continuationCallback(result == MessageBoxResult.Yes);
}
else
{
    // 변경사항 없으면 바로 진행
    continuationCallback(true);
}
}

// 네비게이션시 데이터 정리, 초기화 등은 OnNavigatedFrom에서 처리 가능
public void OnNavigatedFrom(NavigationContext navigationContext)
{
    // 필요하면 작업
}

public bool IsNavigationTarget(NavigationContext navigationContext) => true;
public void OnNavigatedTo(NavigationContext navigationContext) { }
}

```

● CRUD 와 INavigationAware 조합

작업	처리 위치	설명
데이터 로딩	OnNavigatedTo	내비게이션 완료 후 초기 데이터 가져오기
뷰 재사용 여부	IsNavigationTarget	다른 파라미터일 경우 새로운 뷰를 생성할지 판단
변경사항 정리	OnNavigatedFrom	화면을 떠날 때 자동 저장, 리소스 해제 등 처리

ViewModelBase -> RunOnUiThread

CancellationTokenSource / CancellationToken 사용법 및 역할

I RunOnUiThread가 필요한 이유

- UI 요소는 UI 스레드에서만 안전하게 수정할 수 있어요.
- WPF, Xamarin, WinForms 같은 UI 프레임워크는 UI 스레드(메인 스레드)만 UI 컨트롤을 직접 건드릴 수 있도록 제한합니다.
- 만약 다른 스레드(백그라운드 스레드)에서 UI를 직접 수정하면 예외가 발생하거나 예기치 않은 동작이 일어날 수 있어요.

II 언제 사용하는가?

- **비동기 작업에서 UI 상태를 변경할 때**
예를 들어, API 호출, DB 조회 등 시간이 오래 걸리는 작업을 Task.Run이나 async/await로 백그라운드에서 실행한 후, UI를 업데이트해야 할 때 RunOnUiThread로 UI 스레드에서 안전하게 작업을 수행합니다.
- **ObservableCollection 같은 UI 바인딩 컬렉션을 수정할 때**
ObservableCollection은 UI 바인딩에 많이 쓰이는데, 이 컬렉션을 UI 스레드가 아닌 다른 스레드에서 Add(), Clear() 등 변경하면 예외가 납니다.
그래서

```
await RunOnUiThread() =>
{
    Roles.Clear();
    Roles.Add(...);
    return Task.CompletedTask;
};
```

이렇게 UI 스레드에서 컬렉션을 수정해야 안전합니다.

II ObservableCollection에서 사용하는 법

- ObservableCollection<T>는 UI와 직접 바인딩되는 컬렉션입니다.
- UI 스레드 외에서 Add(), Clear(), Remove() 같은 변경 작업을 하면 예외가 발생할 수 있어요.

- 따라서 UI 스레드에서 변경해야 안전합니다.

```
private async void LoadData()
{
    var items = await _repository.GetItemsAsync();

    await RunOnUiThread(() =>
    {
        Items.Clear(); // UI 스레드에서 안전하게 컬렉션 초기화
        foreach (var item in items)
        {
            Items.Add(item); // UI 스레드에서 추가
        }
        return Task.CompletedTask;
    });
}
```

사용예

II IsBusy 같은 단순 바인딩 속성에서 사용법

- IsBusy는 보통 UI에서 로딩 표시를 켜고 끌 때 사용합니다.
- 이 값이 변경되면 UI에 즉시 반영돼야 하므로 UI 스레드에서 변경하는 게 안전합니다.
- 단, 일반적으로 UI 바인딩 속성 변경은 UI 스레드에서만 해야 합니다.

```
await RunOnUiThread(() =>
{
    IsBusy = true; // UI 스레드에서 상태 변경
    BusyMessage = "데이터를 불러오는 중...";
    return Task.CompletedTask;
});
```

사용예

III 기타 다른 바인딩 속성 변경 시

- string, int, bool 같은 단순 타입의 ViewModel 속성도 UI와 바인딩되어 있으면
- 비동기 작업 후 UI 스레드에서 안전하게 변경하는 게 좋습니다

```
await RunOnUiThread(() =>
{
    StatusMessage = "작업 완료";
    ErrorCount = 0;
});
```

사용예

```
return Task.CompletedTask;
});
```

● 왜 RunOnUiThread 을 사용해야 할까?

- WPF(또는 유사 UI 프레임워크)는 UI 스레드만 UI 관련 개체를 변경하도록 제한합니다
- 비동기 작업은 보통 백그라운드 스레드에서 실행되기 때문에,
- 그 결과를 UI에 반영할 때는 반드시 UI 스레드로 전환해야 합니다.
- RunOnUiThread는 이런 전환을 편리하게 처리하는 헬퍼 역할입니다

● 정리

대상	변경 작업 예시	RunOnUiThread 사용 여부
ObservableCollection	Add, Remove, Clear	필수! UI 스레드 안전하게 변경
IsBusy, BusyMessage 등	단순 속성 변경	권장! UI 스레드에서 변경
기타 바인딩 속성	string, int, bool 등 단순 속성 변경	권장! UI 스레드에서 변경 권장! UI 스레드에서 변경

ViewModelBase -> CancellationToken

CancellationTokenSource / CancellationToken 사용법 및 역할

I 개념 요약

- CancellationTokenSource(CTS)는 비동기 작업 취소 요청을 제어하는 개체입니다.
- CancellationToken은 CTS가 생성한 취소 신호 토큰으로, 실제 비동기 작업에서 이 토큰을 보고 취소 여부를 판단합니다.
- 작업이 너무 오래 걸리거나, 사용자가 중단 요청을 할 때 취소 기능을 지원하기 위해 사용합니다.

II 코드 정리 기능 -> ViewModelBase 에서 작성

```
private CancellationTokenSource _cts = new();
```

```
protected CancellationToken CancellationToken => _cts.Token;
```

```
protected void CancelCurrentTask()
```

```
{
```

```
    if (_cts.IsCancellationRequested)
```

```
        return;
```

```
    _cts.Cancel(); // 현재 진행 중인 작업 취소 요청 보내기
```

```
    _cts.Dispose(); // 이전 CancellationTokenSource 자원 해제
```

```
    _cts = new CancellationTokenSource(); // 새로운 토큰 생성 (다음 작업을 위해)
```

```
}
```

- _cts는 현재 진행 중인 작업을 취소할 수 있는 수단
- CancellationToken은 비동기 작업 실행 시 전달하는 취소 신호
- CancelCurrentTask() 호출 시 기존 작업을 취소 요청하고, 새 작업 준비

III 언제, 어떻게 사용하는가?

- 비동기 작업 시작 전, 기존 작업 취소 필요할 때
 - 예: 사용자 입력이 빠르게 바뀌거나, 페이지가 전환될 때
 - 이전 작업이 끝나지 않았으면 취소 요청 후 새 작업 시작

```
protected async Task LoadDataAsync()
{
    CancelCurrentTask(); // 기존 작업이 있으면 취소 요청

    try
    {
        var data = await _repository.GetDataAsync(Cancellation.Token);
        // 작업 성공 후 UI 업데이트
    }
    catch (OperationCanceledException)
    {
        // 취소된 경우 처리 (필요하면 무시해도 됨)
    }
}
```

- ViewModel에서 Navigation 전환 시 작업 취소
 - 사용자가 화면을 떠날 때, 진행 중인 작업을 취소하여 리소스 낭비 방지

```
public override void OnNavigatedFrom(NavigationContext navigationContext)
{
    CancelCurrentTask(); // 진행 중인 작업 즉시 취소
}
```

- UI 버튼, 명령어로 작업 취소
 - 사용자가 "취소" 버튼 누르면 취소 신호 전달

```
private DelegateCommand _cancelCommand;
public DelegateCommand CancelCommand =>
    _cancelCommand ??= new DelegateCommand(() => CancelCurrentTask());
```

IV Cancellation.Token을 사용하는 비동기 메서드 예시

```
public async Task<IEnumerable<Role>> GetAllAsync(Cancellation.Token token)
{
    // 데이터 가져오기 시 취소 가능하도록 토큰을 전달
    return await _dbContext.Roles.ToListAsync(token);
}
```

V 요약

시나리오	사용 방법	목적
새 비동기 작업 시작 시	CancelCurrentTask() 호출 후 시작	이전 작업 즉시 취소
화면 전환(네비게이션) 시	OnNavigatedFrom()에서 호출	작업 중단, 리소스 해제
취소 버튼 클릭 시	취소 명령에서 호출	사용자 요청에 의한 작업 중단
비동기 작업 내부	CancellationToken 사용	작업 취소 신호 감지/종료 처리